A Scientific Concept for Agentic Informal Theorem Assistance over Mathematical Knowledge Bases

Neuro-symbolic AI Group
Idiap Research Institute

March 17, 2026

Abstract

This document presents the scientific concept behind the Abducer framework: an agentic, knowledge-base-centered approach to informal theorem assistance over mathematical source material expressed in natural language and LaTeX. The framework is designed for hard mathematical settings in which direct formal proof is unavailable, incomplete, or strategically premature. Instead of aiming first at proof certification, it constructs a typed web of statements, concepts, methods, barriers, hypotheses, proof obligations, queued conjectures, and persistent proof artifacts, then applies distinct inference modes to organize, connect, test, revise, prove, govern, and interpret that web. The central scientific claim is that mathematical discovery can be operationalized as controlled movement over a structured epistemic object: a problem-specific knowledge base whose theorem-body grounding, coherence, explanatory power, conceptual bridge structure, proof-sketch viability, and outer-loop proof state can all be made explicit, inspectable, and revisable. The approach therefore treats informal theorem assistance not as one-shot question answering, but as an iterative program of theorem-body normalization, concept formation, bridge-object conjecture, proof-queue construction, critique-governed prove cycles, theory-level coherence management, and meta-level reflection.

1 Introduction

Hard mathematical problems rarely fail because isolated statements are unknown. They fail because the available concepts, representations, and methods do not yet cohere into a route that transports local knowledge into a global proof. In such settings, a useful reasoning system must do more than retrieve facts or restate known theorems. It must help identify where the inferential web is sparse, where bridge objects are missing, which conceptual interfaces are overburdened, and which proof moves are blocked by structural barriers.

The Abducer framework is built for that setting. It takes mathematical source material in LaTeX, parses it into a problem-specific knowledge base (KB), and then applies a family of agentic skills that operate over the KB through distinct inference modes. The aim is not to replace formal theorem proving, but to provide an intermediate scientific layer between informal mathematical literature and eventual formal proof: a layer in which hypotheses, conceptual shifts, bridge objects, proof sketches, queued conjectures, proof attempts, and theory revisions can be generated, tested, logged, interpreted, and compared.

2 Scientific Objective

The framework targets the following scientific problem:

How can an agentic reasoning system support mathematical discovery in domains where direct proof is not yet available, by explicitly representing and iteratively revising the conceptual and inferential structure of the problem?

This objective differs from classical automated theorem proving in three ways.

1. It treats *problem understanding* as a first-class computational object.
2. It evaluates not only statements, but also *connections*, *barriers*, *representational shifts*, *intermediate invariants*, and *bridge abstractions*.
3. It makes the reasoning process itself inspectable through typed artifacts, run-scoped logs, human-readable reports, graph views, and publication-quality documents.

3 Phase 0: Theorem-Body Normalization and Claim Licensing

The current framework starts one step earlier than a bridge or proof-sketch pipeline. Before discovery-stage ranking, it requires a normalization pass that turns raw statements into theorem-body form and records what is actually licensed to be asserted.

Definition 1 (Theorem-body normalization)**.** *Given a raw statement excerpt s, the normalization operator produces*

$$\mathcal{N}(s) = (H_s, C_s, Q_s, M_s, T_s),$$

where H_s is the set of typed hypotheses, C_s the typed conclusion, Q_s the quantifier regime, M_s the ambient model and admissibility assumptions, and T_s the threshold or asymptotic conditions.

This step matters scientifically because bridge planning over topic labels is not yet mathematical reasoning. The framework therefore introduces an explicit assertion license for every live claim.

Definition 2 (Claim class)**.** *Every important emitted claim should be readable as exactly one of:*

- *grounded local theorem;*
- *conditional route claim;*
- *conjectural bridge-object proposal;*
- *research recommendation.*

This discipline is what prevents coherence, novelty, and structural fit from masquerading as evidence.

4 Knowledge Base as the Central Scientific Object

The central object of the framework is a one-file mathematical KB. For a given problem, the KB stores canonical statements, definitions, axioms, theorems, barriers, and open gaps in a common structure. The KB is not just a repository of facts. It is the evolving scientific state of the problem.

Definition 3 (Problem KB)**.** *For a given mathematical domain, a KB state at reasoning step t is a typed structure*

$$\mathcal{K}_t = (S_t, C_t, M_t, B_t, R_t, H_t, O_t, A_t),$$

where:

- S_t *is the set of statements;*
- C_t *is the set of concepts;*
- M_t *is the set of methods or proof regimes;*
- B_t *is the set of barriers or obstructions;*
- R_t *is the set of typed relations among these objects;*
- H_t *is the current hypothesis set;*
- O_t *is the set of proof obligations and unresolved gaps;*
- A_t *is the artifact history generated by the reasoning process.*

This representation allows the system to operate on multiple levels simultaneously:

- statement level: what is known or claimed;
- concept level: which mathematical objects and abstractions are active;
- method level: which proof regimes or representations are in use;
- barrier level: what blocks transfer from one region of the KB to another;
- hypothesis level: what new inferential moves are being proposed.

5 Inference Modes as Scientific Operations

The framework is organized around distinct inference modes. Each mode is a scientific operation over the current KB state rather than a generic language-model prompt.

5.1 Cohere

`cohere` is the entry mode for relational formation. It combines connection analysis with coherence evaluation. Its purpose is to identify high-information interfaces, plausible bridge relations, and the current global fit of the theory web.

The key scientific role of this mode is to move from a flat collection of statements to an explicit inferential graph. Without this step, later reasoning is forced to operate on local textual adjacency rather than theory structure.

5.2 Explain

`explain` produces grounded interpretation over the KB. It does not generate new inferential content; it stabilizes intelligibility. In a scientific workflow this matters because discovery must remain auditable by a human mathematician.

5.3 Deduce

`deduce` derives conservative consequences from existing material. It is intentionally narrow. Its role is to tell us which consequences are already licensed by the KB and which proposed bridges exceed the currently available inferential support.

5.4 Abduce

`abduce` hypothesizes missing premises, missing interfaces, or latent lemmas that would make a desired connection go through. In the framework, abduction is not free association. It is constrained by the current KB structure and by the requirement that hypotheses become explicit objects for later testing.

5.5 Conjecture

`conjecture` is a distinct mode that investigates concept gaps and proposes the types of mathematical objects that could fill them. This is one of the key scientific innovations of the framework. The conjecture mode does not merely ask which statement is missing; it asks what *kind of abstraction* is missing.

Examples of conjectured bridge objects include:

- representation shifts;
- intermediate invariants;
- definition introductions;
- barrier-escape principles;
- transfer objects between regimes;
- normal forms that preserve inferentially relevant structure.

5.6 Proof-Sketch

`proof-sketch` takes conjectures, bridge plans, and strategies, and turns them into structured proof-outline programs. It is not a proof checker. It is a mechanism for articulating candidate proof architecture: which lemmas are needed, which invariants are preserved, where barriers appear, and where the route could fail.

5.7 Induce

`induce` is kept only as an optional late-stage synthesis mode. It is not treated as a primary discovery mode over raw KB content. Its role is to generalize already-supported bridge, strategy, or proof-sketch patterns into reusable schemas.

5.8 Discovery Cycle

`kb-discovery-cycle` turns the strongest current routes into a persistent proof queue. This is a scientific shift from open-ended brainstorming to theorem-shaped frontier management. Each queue item is a self-contained conjecture, paired with a route lemma, candidate carrier, invariant, and failure logic.

5.9 Prove Cycle

`kb-prove-cycle` consumes exactly one queued conjecture and turns it into an auditable informal proof or disproof attempt. It is local-first by design: it checks the KB, current artifacts, and problem-scoped MathLib before web search is even eligible. It also makes critique explicit through

judges for grounding, inferential validity, assumption discipline, citation fidelity, proof granularity, local counterexample search, obstruction analysis, and closure.

5.10 Work and Meta

`work` is the stop-aware outer loop that alternates discovery and prove while carrying persistent work-state, proof debt, and plausibility signals. `meta` is the reflection layer: it asks why the problem remains hard, how the proof state should be interpreted, and what conceptual understanding has shifted or persisted. Together they turn the system from a one-pass artifact factory into a governed research process.

6 A Coherentist View of Mathematical Discovery

The framework adopts a coherentist orientation. Individual hypotheses are not judged only in isolation. They are judged relative to the evolving web of claims, concepts, methods, and barriers.

Definition 4 (Typed coherence graph)**.** *Given a KB state $\mathcal{K}_t$, the coherence layer constructs a graph*

$$G_t = (V_t, E_t, \tau_t, \omega_t),$$

where V_t are typed nodes, E_t are typed edges, τ_t maps nodes and edges to semantic types, and ω_t assigns weights reflecting support, attack, explanation, reframing, or blockage.

The purpose of this layer is not philosophical ornament. It is operational. A hypothesis can be locally plausible yet globally damaging if it introduces tension, redundancy, or barrier conflict elsewhere in the graph. Conversely, a hypothesis can deserve promotion not because it is deductively complete, but because it increases theory-wide explanatory fit and aligns multiple previously disconnected regions.

The framework therefore uses both local and global measures:

- local support and refutation signals;
- local explanatory power;
- change in global coherence under acceptance;
- interaction with existing barriers;
- downstream effect on future deduction, abduction, and bridge planning.

7 Persistent Proof Memory and Search Governance

The framework now includes two persistent memory objects beyond the core KB.

Definition 5 (Proof queue)**.** *For problem instance P, the proof queue*

$$\mathcal{Q}_P = \{q_1, \ldots, q_n\}$$

stores self-contained conjectures selected for proof or disproof. Each q_i includes the route claim, active carrier, invariant, route lemma, and current status.

Definition 6 (Problem-scoped MathLib)**.** *For problem instance P, the MathLib*

$$\mathcal{M}_P = (\mathcal{R}_P, \mathcal{E}_P)$$

stores locally produced results $\mathcal{R}_P$ together with imported external theorems, definitions, and axioms $\mathcal{E}_P$ that were actually used in proof attempts.

This matters because the framework no longer reasons only by regenerating intermediate prose. It accumulates route-level memory. Negative results, prior-art findings, and standalone subresults can therefore become persistent parts of the research state rather than transient by-products.

The governance layer is equally important. Discovery is now explicitly controlled by anti-drift criteria, conjecture-value rubrics, revisit penalties, and end-to-end proof critics. In scientific terms, the system is evolving from an exploratory concept engine into a governed research process in which depth must be justified by theorem-body gain, typed carriers, sharper falsifiers, or independent subresult value.

8 Conceptual Specificity as a Requirement for Mathematical Reasoning

A central design principle of the framework is that mathematical concepts must be represented with enough specificity to preserve inferential identity. Broad topical labels such as "gauge theory" or "circuit complexity" are useful for navigation, but they are not sufficient for reasoning.

Principle 1 (Inferential specificity)**.** *A concept representation is specific enough for mathematical reasoning only if two statements with the same representation are interchangeable for the next intended inferential move.*

This principle forces the framework to model not just topics, but structured signatures such as:

- antecedent schema;
- target consequence schema;
- preserved invariant;
- barrier scope;
- model or regime;
- transfer mechanism;
- proof obligation type.

The practical effect is that the framework can distinguish between conceptually adjacent but inferentially non-equivalent statements. That distinction is necessary if the system is to act as an informal theorem assistant rather than as a thematic clustering tool.

9 Bridge Objects and Concept-Gap Conjecture

In difficult mathematics, progress often depends on the discovery of the right intermediate object rather than on a direct derivation from known premises to the target theorem. The framework therefore introduces a dedicated conjecture stage for *bridge objects*.

A bridge object is any mathematical construct whose introduction makes a previously weak or impossible transfer become structured and testable. Examples include:

- a witness that survives translation between formulations;
- an invariant that persists across a reduction or reconstruction;
- a representation in which a barrier becomes visible and attackable;
- a surrogate problem whose resolution would imply the original target;
- a normal form that isolates the proof-relevant structure of a class of statements.

The conjecture layer also carries two additional reasoning devices.

9.1 Counterfactual reasoning

For a candidate bridge object, the system asks what breaks if a proposed attribute is removed, relaxed, or replaced. This helps separate essential structure from incidental description.

9.2 Definitional reasoning

The framework attempts to articulate a *definitional profile* for the bridge object:

$$D(h) = (N(h), S(h), X(h)),$$

where:

- $N(h)$ is the set of necessary attributes;
- $S(h)$ is the family of sufficient attribute sets;
- $X(h)$ is the set of superficial or excluded attributes.

This matters scientifically because a bridge object should not merely be named; it should be defined in a way that exposes why it would carry inferential force.

10 Proof Sketching as Structured Scientific Hypothesizing

Proof sketches in the framework are not free-form narrative drafts. They are typed artifacts that expose:

- the strategy family;
- the sequence of proof obligations;
- the required lemmas;
- preserved invariants;
- barrier checkpoints;
- abort conditions;
- counterfactual failure tests.

This turns proof sketching into a scientific instrument. A sketch can fail productively. Even when it does not resolve the problem, it clarifies where the missing mathematics lies.

The framework currently supports several proof-sketch strategy families, including:

- invariant lift through intermediate object;
- surrogate reduction transfer;
- barrier-escape re-encoding;
- contradiction via collapse or separation;
- bootstrapped derandomization or transfer;
- regime-switch case analysis.

11 Hypothesis Evaluation and Refutation Loops

The framework does not stop at generating hypotheses. It evaluates them. Each hypothesis receives quantitative scores for support, refutation, and confidence, as well as a typed inferential readout.

For a hypothesis h, the evaluator computes at least:

- structural compatibility between support and target statements;
- grounding of the antecedent schema;
- grounding of the target consequence;
- grounding of the preserved invariant;
- grounding of the barrier scope;
- local explanatory power and coherence effects;
- evidence adjustments from deduction, abduction, or contradiction signals.

The result is a verdict such as supported, refuted, or inconclusive. Inconclusive is a first-class outcome. This is important because open mathematical discovery should preserve uncertainty rather than force premature binary decisions.

The current framework additionally distinguishes claims that are only *internally supported*. This class captures an important epistemic fact: a route can fit the current structural scoring regime without yet carrying genuine external or adversarial support.

The framework also supports iterative refutation loops. Unresolved hypotheses can be retested in later rounds as the KB changes. If the theory stagnates, a revision loop can propose ontology repair, concept splitting, or re-encoding of the problem.

12 Observability and Human Interpretability

One of the framework's scientific commitments is that the reasoning process itself must remain interpretable. For that reason, every substantial stage emits:

- structured JSON artifacts for machine reuse;
- markdown reports for human inspection;
- run-scoped reasoning logs with agent names and stage outputs;
- graph-based process views with typed edges;

- concept-evolution summaries;
- persistent proof queue, work-state, and plausibility artifacts;
- meta-reflection reports and expert-facing slide narratives;
- publication-oriented LaTeX and PDF exports for hypothesis and review documents.

This observability layer serves two purposes. First, it enables debugging and critical review of the reasoning process. Second, it makes the system usable as a scientific collaborator rather than as a black box.

13 Illustrative Use Cases

The framework has been exercised on at least two hard-problem KBs:

- **P vs NP**: where the system concentrates on theorem-family interfaces such as hardness magnification, proof complexity, derandomization, and meta-complexity;
- **Yang–Mills existence and mass gap**: where the system surfaces interfaces among Euclidean OS data, Wightman reconstruction, transfer matrices, correlator decay, spectral gap witnesses, and continuum-limit barriers.

These examples illustrate the intended scope of the framework: domains in which the main challenge is not recalling isolated facts, but constructing a conceptually coherent route through multiple mathematical regimes.

14 Why This Approach Is Scientifically Useful

The approach is scientifically useful because it produces explicit intermediate objects of inquiry. Instead of jumping directly from problem statement to proof attempt, it yields:

- ranked intervention points;
- explicit bridge hypotheses;
- conjectured abstraction types;
- proof strategies;
- proof sketches;
- queued theorem-shaped proof targets;
- persistent proof attempts and MathLib entries;
- evaluated and iterated hypothesis sets;
- theory-level coherence reports;
- end-to-end plausibility and governance reports.

These outputs support a mode of research in which mathematical discovery becomes inspectable, revisable, and strategically organized. The system can therefore support not only answer generation, but problem decomposition, research planning, conceptual diagnosis, and the disciplined exploration of alternative proof pathways.

15 Limitations

The framework is deliberately not a full formal prover. Its current limitations include:

- dependence on KB quality and statement granularity;
- persistent risk that coherence or novelty can still be mistaken for evidence if the claim-class discipline is ignored;
- residual heuristic elements in concept extraction and bridge typing;
- only partial closure of the revision loop into executable KB transformations;
- limited direct theorem validation compared with proof assistants;
- possible overproduction of conceptually appealing but mathematically weak hypotheses if specificity constraints are not enforced;
- prove-cycle outputs that are still informal arguments rather than certified proofs.

These limitations are acceptable only if they remain visible and operationally constrained. That is why the system emphasizes logs, typed commitments, coherence scoring, and explicit falsifiers.

16 Conclusion

The Abducer framework embodies a scientific concept of informal theorem assistance grounded in structured KB evolution rather than one-shot theorem solving. Its core thesis is that hard mathematical reasoning can be productively supported by making theorem bodies, concepts, barriers, bridge objects, coherence relations, proof obligations, queued conjectures, and proof-state governance computationally explicit. In this view, the central achievement of the system is not merely to generate candidate proofs, but to transform a difficult mathematical problem into an inspectable evolving epistemic object.

That transformation is the main scientific contribution of the approach. It creates a disciplined intermediate layer between informal literature and formal proof, and it gives both human researchers and agentic skills a common state on which conceptual discovery, abductive repair, coherentist evaluation, and proof planning can proceed.